UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA — PROGRAMACIÓN II

Subnotway: Pan, Puntos y Premios

Guía Técnica Integral para la Defensa y Presentación del Proyecto

Spring Boot 3.4 | Java 17+ | MySQL (InnoDB) | OpenPDF | Android Nativo (Java) | Thymeleaf | Bootstrap 5 | REST API

1 Resumen Ejecutivo (El "Elevator Pitch")

> *"Subnotway es una solución tecnológica integral orientada a optimizar la operación comercial y de inventarios de un restaurante de comida rápida, complementada con un sistema de lealtad y retención de clientes. Consta de dos componentes clave totalmente desacoplados: una **plataforma web administrativa y de punto de venta (POS)** construida sobre Spring Boot y Thymeleaf, y una **aplicación móvil Android nativa** que permite a los clientes consultar su saldo de puntos en tiempo real a través de una API REST segura con persistencia transaccional en MySQL."*

1. Terminal POS

Toma de pedidos reactiva con catálogo, control de cambio, acumulación automática de puntos y estados (`PAGADO` → `ENTREGADO`).

2. Inventario y Kardex

Control de 29 productos oficiales, alertas visuales de stock crítico (≤ 5 unidades) y registro de auditoría de movimientos.

3. Directorio de Clientes

Gestión completa con soporte para identificación guatemalteca auténtica (NIT con/sin guion o DPI de 13 dígitos).

4. Fidelización y Canjes

Consulta de saldo, historial de compras y canje atómico de premios (10, 15, 30 y 50 pts) con validación estricta de stock.

5. Centro de Reportes PDF

Generación en memoria mediante OpenPDF de 6 reportes gerenciales con filtros de fecha dinámicos y cero residuos en disco.

6. Componente Móvil Android

App nativa desacoplada que consulta saldo en vivo mediante peticiones REST multihilo sin bloquear la interfaz de usuario.

2 Arquitectura de Software y Patrones de Diseño



Patrones de Diseño Implementados:

- **MVC (Modelo-Vista-Controlador):** Separación absoluta entre vistas HTML, lógica de control y modelos de dominio.
- **DAO (Data Access Object):** Encapsula todas las sentencias SQL y la conectividad JDBC. Si cambia la base de datos, los servicios no sufren cambios.
- **Service Layer:** Controladores ligeros (*Thin Controllers*) que delegan la lógica transaccional a servicios especializados.

Principios y Buenas Prácticas:

- **REST (Representational State Transfer):** Endpoints sin estado (*stateless*) que devuelven JSON estándar consumible por cualquier cliente.
- **Concurrencia Multihilo (Android):** Red ejecutada en `ExecutorService` y renderizado en UI mediante `Handler(Looper.getMainLooper())`.
- **DTO (Data Transfer Object):** `ItemPedidoDTO` transfiere las líneas del carrito evitando sobrecargar entidades.

3 Explicación del Código Fuente Sección por Sección

A. Módulo de Ventas y Punto de Venta (POS)

Ubicación: `controlador/VentaController.java` | `servicio/VentaServicio.java` | `dao/impl/PedidoDAO.java`

1 **Recepción de Datos y Parsing DTO:**
El navegador envía por `POST /ventas/procesar` los datos del cliente, método de pago (`EFECTIVO` / `TARJETA`), monto recibido y un payload JSON con las líneas del carrito. En `VentaServicio.procesarVenta` se deserializa en objetos `ItemPedidoDTO` .

2 **Validaciones Financieras y Descuento de Stock:**
Se valida que `montoRecibido ≥ total` y se calcula `cambio = montoRecibido - total` . Se descuenta el stock físico con `productoDAO.descontarStock` . Si el ítem es un combo, se consultan sus componentes en `menu_componentes` y se descuenta individualmente cada ingrediente.

3 **Persistencia Transaccional y Auditoría:**
Se inserta el pedido con estado `'PAGADO'` , se guardan los detalles en `pedido_detalles` , el registro contable en `pagos` , se calcula la bonificación de puntos al cliente y se registran auditorías en `movimientos_puntos` y `movimientos_inventario` .

4 **Transición de Entrega:**
Cuando el pedido es preparado y entregado, el operario hace clic en "Entregar" ejecutando `POST /ventas/entregar/{id}` , lo que actualiza el estado a `'ENTREGADO'` en la base de datos.

B. Módulo de Fidelización y Canje Atómico de Premios

Ubicación: `controlador/FidelizacionController.java` | `servicio/FidelizacionService.java` | `dao/impl/RecompensaDAO.java`

1. Fórmula Oficial de Puntos:

```
public int calcularPuntos(int sandwiches, int menus) {
    return (sandwiches * 2) + (menus * 8);
}
```

Cada sándwich individual otorga **2 puntos** y cada menú/combo completo otorga **8 puntos** (estrategia de negocio para incentivar ventas de mayor margen).

2. Transacción de Canje en Cascada: En `FidelizacionService.procesarCanje(idCliente, idRecompensa)` se realizan 5 validaciones rigurosas antes de alterar cualquier tabla:

1. **Cliente Activo:** Valida que el cliente exista y su estado sea `'ACTIVO'` (rechaza a clientes inactivos como Consumidor Final).
2. **Recompensa Activa:** Valida que el premio solicitado esté habilitado en el catálogo.
3. **Saldo Suficiente:** Valida que `cliente.getPuntos() ≥ recompensa.getCostoPuntos()` .
4. **Stock Físico Garantizado:** Verifica que haya existencias disponibles del producto o de los componentes del combo.
5. **Descuento Atómico:** Descuenta los puntos al cliente, descuenta el inventario y genera el registro en `canjes` y auditorías correspondientes.

C. Centro de Reportes PDF con OpenPDF (In-Memory Streaming)

Ubicación: `controlador/ReporteController.java` | `servicio/ReportePdfService.java` | `dao/impl/ReporteDAO.java`

¿Cómo se genera el archivo PDF sin guardarlo en el disco duro?

1. El controlador define los encabezados HTTP: `Content-Type: application/pdf` y `Content-Disposition: inline; filename=Reporte.pdf`.
2. Pasa el flujo de salida del servlet `response.getOutputStream()` directamente a OpenPDF.
3. `ReportePdfService` inicializa un `Document(PageSize.A4, 36, 36, 36, 36)` y conecta `PdfWriter.getInstance(document, outputStream)`.
4. Aplica tablas `PdfPTable` con celdas estilizadas, encabezados verde institucional (`#008938`) y *Zebra Striping* (filas intercaladas).
5. Al invocarse `document.close()`, el archivo binario viaja en tiempo real hacia el navegador del cliente. Cero consumo de disco y mínima huella en memoria RAM.

Consulta Relacional Avanzada con `UNION ALL` (Productos Más Vendidos):

```sql
SELECT prod.codigo, prod.nombre, prod.categoria, SUM(t.unidades) AS total_unidades
FROM (
    -- 1. Unidades vendidas de forma individual
    SELECT pd.id_producto, pd.cantidad AS unidades
    FROM pedido_detalles pd JOIN pedidos p ON pd.id_pedido = p.id_pedido
    WHERE pd.id_producto IS NOT NULL AND p.estado IN ('PAGADO', 'ENTREGADO', 'COMPLETADO')

    UNION ALL

    -- 2. Unidades consumidas dentro de combos (multiplicando combos x componentes)
    SELECT mc.id_producto, (pd.cantidad * mc.cantidad) AS unidades
    FROM pedido_detalles pd JOIN pedidos p ON pd.id_pedido = p.id_pedido
    JOIN menu_componentes mc ON pd.id_menu = mc.id_menu
    WHERE pd.id_menu IS NOT NULL AND p.estado IN ('PAGADO', 'ENTREGADO', 'COMPLETADO')
) t
JOIN productos prod ON t.id_producto = prod.id_producto
GROUP BY prod.id_producto, prod.codigo, prod.nombre, prod.categoria
ORDER BY total_unidades DESC;
```

D. Aplicación Móvil Android Nativa (Java)

Ubicación: `MainActivity.java` | `res/layout/activity_main.xml` | `AndroidManifest.xml`

¿Cómo funciona la asincronía y por qué es fundamental?

- **Worker Thread (ExecutorService):** En Android es una regla obligatoria no ejecutar operaciones de red en el hilo principal para evitar el error fatal `NetworkOnMainThreadException`. Se envía la solicitud HTTP hacia `http://10.0.2.2:8080/api/clientes/puntos/{identificador}` mediante un hilo en segundo plano.
- **UI Thread (Handler con Looper.getMainLooper):** Una vez recibida la cadena JSON y parseada con `JSONObject`, se delega la actualización visual a la cola del hilo principal mediante `mainHandler.post(() -> { ... })`, garantizando una interfaz fluida sin bloqueos.

E. Capa de Persistencia y Seguridad (JDBC DAO)

- **Mitigación de SQL Injection:** El 100% de las consultas utilizan `PreparedStatement` con parámetros tipificados (`setString`, `setInt`, `setTimestamp`). El motor compila la estructura de la consulta antes de enlazar datos, eliminando cualquier riesgo de inyección.
- **Búsqueda Inteligente de Clientes:** `ClienteDAO.buscarPorIdentificador` normaliza las entradas permitiendo buscar por NIT con guion (`87654321-0`), NIT sin guion (`876543210`), DPI de 13 dígitos (`2548123450101`) o ID numérico/código legado (`CL-002`).

4 Directorio Oficial de Clientes (NIT y DPI)

Tabla de datos precargados para demostración durante la evaluación:

ID	NIT / DPI	Nombre del Cliente	Puntos	Estado	Caso de Uso para Demostración
1	`12345678-9`	**Juan Carlos Pérez**	**1,316**	**ACTIVO**	Cliente VIP con puntos acumulados suficientes para probar canjes en vivo.
2	`87654321-0`	**María Fernanda Gómez**	**240**	**ACTIVO**	*(Reemplazo de CL-002)*. Cliente frecuente; ideal para consulta en Android.
3	`2548123450101`	**Carlos Eduardo López**	**85**	**ACTIVO**	Demostración de búsqueda por **DPI (13 dígitos)**.
4	`45678912-3`	**Ana Lucía Morales**	**52**	**ACTIVO**	Umbral exacto para canjear **Menú Completo (50 pts)**.
5	`1892345670101`	**Roberto Alejandro Soto**	**38**	**ACTIVO**	Umbral exacto para canjear **Sándwich (30 pts)**.
6	`98765432-1`	**Sofía Isabel Martínez**	**18**	**ACTIVO**	Umbral exacto para canjear **Acompañamiento (15 pts)**.
7	`3012456780101`	**Diego Fernando Castillo**	**12**	**ACTIVO**	Umbral exacto para canjear **Bebida Gratuita (10 pts)**.
8	`54321678-K`	**Jimena Alejandra Valdez**	**0**	**ACTIVO**	Demuestra validación de **saldo insuficiente** en canje.
9	`2145678900101`	**César Augusto Ramírez**	**320**	**ACTIVO**	Cliente corporativo con saldo intermedio.
10	`CF`	**Consumidor Final**	**0**	**INACTIVO**	Demuestra validación de **cliente inactivo** para fidelización.

5 Preguntas "Trampa" de Catedráticos y Respuestas Ideales

P1: "¿Por qué la aplicación móvil no se conecta directamente a MySQL con JDBC?"

Respuesta Técnica: Conectar una app móvil directamente a la base de datos es una grave vulnerabilidad de seguridad. Requeriría incluir credenciales de MySQL dentro de un APK descompilable, obligaría a abrir el puerto 3306 a internet exponiendo la base de datos a ataques externos, y saturaría el pool de conexiones. Al usar una **API REST intermedia**, desacoplamos la aplicación cliente, centralizamos la autenticación y validamos todas las reglas de negocio en el backend.

P2: "¿Dónde reside la lógica de negocio: en la base de datos, en el controlador o en el servicio?"

Respuesta Técnica: En la **capa de servicio (** `FidelizacionService` **,** `VentaServicio` **,** `ReportePdfService` **)**. Los controladores son delgados (*Thin Controllers*) y únicamente gestionan peticiones HTTP. Tampoco sobrecargamos la base de datos con stored procedures complejos para preservar la testeabilidad unitaria en Java y mantener la independencia del motor SQL.

P3: "¿Qué sucede si dos usuarios intentan canjear puntos simultáneamente?"

Respuesta Técnica: El motor de almacenamiento **InnoDB** implementa bloqueo a nivel de fila (*Row-Level Locking*) y transacciones con propiedades ACID. En la capa de servicio se realiza una validación atómica de saldo y existencias antes de la modificación física, garantizando la consistencia del saldo.

P4: "¿Por qué usar OpenPDF en lugar de iText antiguo o librerías de terceros pesadas?"

Respuesta Técnica: OpenPDF es una bifurcación moderna de código abierto bajo licencias LGPL y MPL, libre de las restricciones y costos comerciales de iText 7. Se integra nativamente con Spring Boot 3 y Java 17+, permitiendo escribir el documento binario directamente sobre el `OutputStream` del servlet sin escribir archivos temporales en el disco.

P5: "¿Qué significa la dirección 10.0.2.2 configurada en la aplicación Android?"

Respuesta Técnica: Es el alias de loopback especial provisto por el emulador de Android. Dentro de la máquina virtual del emulador, `127.0.0.1` apunta a sí mismo; para alcanzar el `localhost` de la máquina anfitriona (donde corre Spring Boot en el puerto 8080), se debe emplear `10.0.2.2` .

6 Guía Paso a Paso para la Demostración en Vivo (Demo)

Sigue este orden secuencial durante la defensa para lograr el máximo impacto técnico y visual:

1 Terminal de Ventas POS (/ventas):

- Selecciona al cliente **Juan Carlos Pérez** (`12345678-9`).
- Agrega **1 Combo 1** (Q 230.00) y **1 Sándwich de Ensalada de Atún** (Q 75.00).
- Ingresa un monto de pago de **Q 350.00** en efectivo y presiona **Cobrar**.
- Señala cómo el sistema calcula el cambio exacto (**Q 45.00**) y acumula **+10 puntos** (8 por el combo + 2 por el sándwich).
- Presiona **Entregar** para evidenciar la transición de estado (`PAGADO` → `ENTREGADO`).

2 Consulta Móvil en Android (Emulador o Dispositivo):

- Abre la aplicación móvil Subnotway.
- Digita el NIT de Juan Carlos: `12345678-9` y presiona **Consultar Saldo**.
- Demuestra cómo los puntos acumulados en la venta recién efectuada se reflejan al instante vía REST API.
- Digita también el NIT de María Fernanda: `87654321-0` (240 pts) para evidenciar la reactividad de la consulta.

3 Módulo de Fidelización Web (/fidelizacion):

- Busca al cliente Juan Carlos Pérez o Carlos Eduardo López.
- Ejecuta el canje de una recompensa (ej. "Bebida Gratuita" por 10 puntos).
- Muestra cómo se descuentan los 10 puntos del saldo del cliente, se descuenta 1 unidad del inventario físico y se inserta la transacción en la tabla histórica de canjes.

4 Centro de Reportes PDF (/reportes):

- **Ventas por Período:** Selecciona el rango del día actual para descargar el PDF con las ventas recién registradas.
- **Stock Crítico / Bajo:** Abre el reporte para mostrar los 5 productos con alertas de stock (Monster rosa en 0, Pork Jhon en 2 unidades).
- **Productos Más Vendidos:** Demuestra el PDF donde se aprecian tanto los productos directos como los ingredientes desagregados de combos mediante la consulta SQL `UNION ALL` .

¡Proyecto 100% Listo y Verificado! Universidad Mariano Gálvez de Guatemala • Facultad de Ingeniería en Sistemas.